

## Kamuoliukų mašina

Madina sukonstravo kamuoliukų mašiną, skirtą IOI dalyvių pramogai. Mašinos vidinė struktūra tokia:

- Mašiną sudaro  $N$  **viršūnių**, numeruojamų nuo 0 iki  $N - 1$ . Viršūnė  $N - 1$  vadinama **šaknimi**.
- Kiekvienam  $0 \leq u < N - 1$ , viršūnės  $u$  **tėvinė viršūnė** yra viršūnė  $P[u]$ , kur  $P[u] > u$ , o viršūnė  $u$  yra viršūnės  $P[u]$  **vaikas**. Šaknis neturi tėvinės viršūnės. Viršūnė, neturinti vaikų, vadinama **lapu**.

Jūs **nežinote** nei  $N$ , nei nė vienos viršūnės  $u$  tėvinės viršūnės  $P[u]$ . Vietoj to, Madina jums nurodo  $M$  – lapų skaičių bei tai, kad lapų numeriai yra  $0, 1, \dots, M - 1$ . Nustatykite kamuoliukų mašinos vidinę struktūrą.

Kiekvienoje mašinos viršūnėje gali tilpti daugiausiai vienas **kamuoliukas**. Kiekvienas kamuoliukas turi savo **vertę**. Vertės yra neneigiami sveikieji skaičiai. Sakysime, kad viršūnė turi vertę  $x$ , jei joje kamuoliukas, kurio vertė  $x$ . Viršūnė yra **užimta**, jei joje kamuoliukas. Kitu atveju viršūnė yra **tuščia**. Pradiniu momentu visos viršūnės tuščios.

Galite atlikti dviejų rūšių operacijas:

attempt to insert a new ball with value X at leaf

- insert( $U, X$ )**: naują kamuoliuką, kurio vertė  $X$ , pamėginti įdėti į lapą  $U$ .
  - Jei lapas  $U$  užimtas, operacija grąžina reikšmę `false` ir kamuoliukas niekur neįdedamas.
  - Jei lapas  $U$  tuščias, kamuoliukas įdedamas viršūnėje  $U$ . Tuomet jis laisvai juda į tėvinės viršūnės tol, kol šie tušti. Kamuoliukas sustoja, kai pasiekia šaknį arba viršūnę, kurios tėvinė viršūnė užimta. Operacija grąžina reikšmę `true`.
- collect()**: jei šaknis tuščia (kas reiškia, mašina yra tuščia), `collect` grąžina tuščią masyvą. Kitu atveju, toliau aprašyta rekursyvi procedūra `traverse` surenka visų šiuo metu mašinoje esančių kamuoliukų vertes į masyvą  $S$ . Iš pradžių, masyvas  $S$  tuščias ir iškviečiama `traverse(N - 1)`.

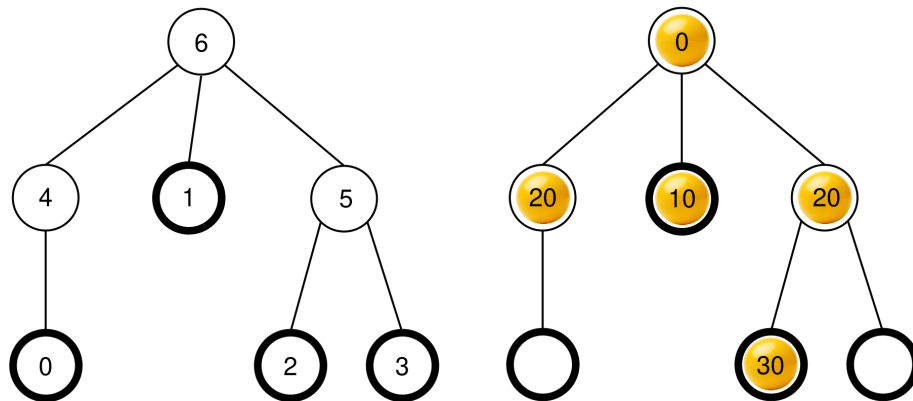
```

traverse(u):
    prideda viršūnėje u esančio kamuoliuko vertę į S galą
    tegu c[u] yra užimtų viršūnės u vaikų sąrašas
    išrikiuoja c[u] nemažėjimo tvarka pagal tose viršūnėse
        esančių kamuoliukų vertes (jei kelios viršūnės turi
        tą pačią vertę, jos gali būti išrikiuotos bet kuria tvarka)
    kiekvienam v iš c[u]:
        traverse(v)

```

Kai ši procedūra baigia darbą, **visi kamuoliukai pašalinami** iš mašinos ir `collect` grąžina  $S$ .

Kaip pavyzdį panagrinėkime mašiną, kuri sudaryta iš  $N = 7$  viršūnių ir kurios tėvinių viršūnių masyvas  $P = [4, 6, 5, 5, 6, 6]$ . Paveiksliukas kairėje vaizduoja tuščią mašiną su skaičiais, žyminčiais viršūnių numerius. Tuo tarpu paveiksliuke dešinėje pavaizduotas vienas iš įmanomų kamuoliukų išdėstymų po tam tikros `insert` operacijų sekos (ši seka pateikta skiltyje Pavyzdys).



Kai `collect()` iškviečiama, šaknis (viršūnė 6) yra užimta, tad  $S$  sukuriamas kaip  $S = []$  ir iškviečiama `traverse(6)`.

**traverse(6):** viršūnės 6 vertė yra 0; ji pridedama į  $S$ . Dabar  $S = [0]$ . Viršūnės 6 vaikai yra viršūnės 4, 1 ir 5. Jie visi užimti ir jų vertės yra 20, 10 ir 20 atitinkamai. Išrikiavus nemažėjimo tvarka gauname  $c[6] = [1, 5, 4]$  (pastebėkime, kad  $c[6] = [1, 4, 5]$  taip pat tiktų, kadangi viršūnės 4 ir 5 turi tą pačią vertę). Procedūra tuomet kviečia `traverse` kiekvienai viršūnei iš  $c[6]$  ta tvarka, kuria jos ten surašytos.

- **traverse(1):** viršūnės 1 vertė yra 10; ji pridedama į  $S$ . Dabar  $S = [0, 10]$ . Viršūnė 1 neturi užimtų vaikų, tad šis iškvietimas daugiau nieko nedaro.
- **traverse(5):** viršūnės 5 vertė yra 20; ji pridedama į  $S$ . Dabar  $S = [0, 10, 20]$ . Viršūnė 5 turi vieną užimtą vaiką: viršūnę 2, tad  $c[5] = [2]$  ir procedūra kviečia `traverse(2)`.
  - **traverse(2):** viršūnės 2 vertė yra 30; ji pridedama į  $S$ . Dabar  $S = [0, 10, 20, 30]$ . Viršūnė 2 neturi užimtų vaikų, tad šis iškvietimas daugiau nieko nedaro.

- Vykdymas grįžta į `traverse(5)`, kur jau išnagrinėti visi vaikai, esantys `c[5]`, tad funkcija daugiau nieko nedaro.
- **`traverse(4)`**: viršūnės 4 vertė yra 20; ji pridedama į  $S$ . Dabar  $S = [0, 10, 20, 30, 20]$ . Viršūnė 4 neturi užimtų vaikų, tad šis iškvietimas daugiau nieko nedaro.

Visi iškvietimai užbaigti ir nebėra daugiau jokių viršūnių, kurias reikia išnagrinėti. Galiausiai kiekvienas kamuoliukas pašalinamas iš mašinos ir `collect` grąžina masyvą  $S = [0, 10, 20, 30, 20]$ .

Jūsų užduotis – išsiaiškinti viršūnių kiekį  $N$  ir rasti tėvinių viršūnių masyvą  $R = [R[0], R[1], \dots, R[N-2]]$ , nusakantį mašinos struktūrą, galimai turinčią skirtingus numerius viršūnių, kurios nėra lapai ar šaknis. Formaliai aprašant, rastas masyvas  $R$  yra laikomas teisingu, jei įmanoma priskirti skirtingus numerius  $L[u]$ , kur  $0 \leq L[u] < N$ , kiekvienai mašinos viršūnei  $u$  taip, kad:

- $L[u] = u$  visiems  $0 \leq u < M$  ir  $u = N-1$ ;
- $L[P[u]] = R[L[u]]$  visiems  $0 \leq u < N-1$ .

**Nėra** reikalaujama, kad  $R[u] > u$ . Mašina **privalo būti tuščia**, kai grąžinsite savo atsakymą.

Tegu  $K$  yra atliktų `collect` operacijų kiekis ir  $B$  yra didžiausia ant kurio nors kamuoliuko užrašyta vertė tarp visų `insert` iškvietimų. Pažymėkime  $C = K + B$ . Tuomet  $C$  **privalo būti ne daugiau kaip** 1000. Jums skiriami taškai kai kuriose dalinėse užduotyse priklauso nuo  $C$  reikšmės.

## Realizacija

Parašykite šią procedūrą:

```
std::vector<int> find_structure(int M)
```

- $M$ : mašinos lapų kiekis.
- Ši procedūra kviečiama lygiai vieną kartą kiekviename teste.
- Ši procedūra turi grąžinti  $N-1$  ilgio masyvą  $R = [R[0], R[1], \dots, R[N-2]]$  of length  $N-1$  – teisingą tėvinių viršūnių masyvą.

Sąveikai su mašina, jūsų procedūra gali kviesti žemiau aprašytas dvi procedūras.

Pirmoji procedūra:

```
bool insert(int U, int X)
```

- $U$ : lapo, į kurį dedamas kamuoliukas, numeris. Turi galioti  $0 \leq U < M$ .
- $X$ : sveikasis skaičius, nusakantis kamuoliuko vertę. Turi galioti  $0 \leq X \leq 1000$ .

- Ši procedūra grąžina reikšmę `true`, jei kamuoliukas sėkmingai įdėtas, ir grąžina reikšmę `false`, jei lapas  $U$  jau užimtas.
- Ši procedūra gali būti iškviesta daugiausiai 500 000 kartų.

Antroji procedūra:

```
std::vector<int> collect()
```

- Surenka visų įdėtų kamuoliukų vertes, išvalo mašiną ir grąžina surinktą masyvą  $S$ .

Jei kuriuo nors jūsų programos vykdymo metu  $C$  viršys 1000, jūsų sprendimas gaus verdiktą `Output isn't correct: Too many resources used`.

Mašinos struktūra užfiksuojama prieš iškviečiant `find_structure`. Vertinimo programa yra **deterministinė**. Tai reiškia, kad paleidus ją du kartus, jei atliekama tokia pati operacijų seka, `collect` iškvietimai grąžins vienodus masyvus.

## Ribojimai

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

## Dalinės užduotys

| Dalinė užduotis | Taškai | Papildomi ribojimai           |
|-----------------|--------|-------------------------------|
| 1               | 5      | Šaknis turi lygiai $M$ vaikų. |
| 2               | 10     | $M \leq 3$                    |
| 3               | 25     | $N \leq 200, M \leq 45$       |
| 4               | 60     | Jokių papildomų ribojimų.     |

3-ojoje ir 4-ojoje dalinėse užduotyse jūsų taškai priklauso nuo  $C$  reikšmės, kaip aprašyta žemiau.

3-oji dalinė užduotis (25 taškai)

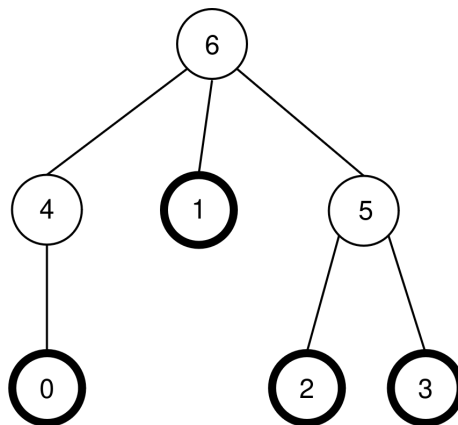
| Ribojimai          | Taškai |
|--------------------|--------|
| $1000 < C$         | 0      |
| $45 < C \leq 1000$ | 13     |
| $C \leq 45$        | 25     |

#### 4-oji dalinė užduotis (60 taškų)

| Ribojimai           | Taškai             |
|---------------------|--------------------|
| $1000 < C$          | 0                  |
| $200 < C \leq 1000$ | 7                  |
| $71 < C \leq 200$   | $47 - \frac{C}{5}$ |
| $44 < C \leq 71$    | $104 - C$          |
| $C \leq 44$         | 60                 |

### Pavyzdys

Panagrinėkime tokią pačią mašiną, kaip aprašyta sąlygoje anksčiau. Turime  $N = 7$ ,  $M = 4$  ir  $P = [4, 6, 5, 5, 6, 6]$ . Ši mašina vėl pavaizduota žemiau esančiame paveikslėlyje:



Vertinimo programa iškviečia:

```
find_structure(4)
```

Ši procedūra gali atlikti tokią iškvietimų seką:

1. **insert(0, 0):** lapas 0 yra tuščias, tad kamuoliukas, kurio vertė 0, įdedamas į lapą 0. Viršūnė  $P[0] = 4$  yra tuščia, tad kamuoliukas pajuda į viršūnę 4. Viršūnė  $P[4] = 6$  yra tuščia, tad kamuoliukas pajuda į viršūnę 6. Kadangi 6 yra šaknis, kamuoliukas daugiau nejuda. Grąžinama reikšmė `true`.
2. **insert(3, 20):** lapas 3 yra tuščias, tad kamuoliukas, kurio vertė 20, įdedamas į lapą 3. Viršūnė  $P[3] = 5$  yra tuščia, tad kamuoliukas pajuda į viršūnę 5. Kadangi  $P[5] = 6$  yra užimta, kamuoliukas daugiau nejuda. Grąžinama reikšmė `true`.
3. **insert(1, 10):** lapas 1 yra tuščias, tad kamuoliukas, kurio vertė 10, įdedamas į lapą 1. Viršūnė  $P[1] = 6$  yra užimta, tad kamuoliukas lieka viršūnėje 1. Grąžinama reikšmė `true`.

4. **insert(2, 30)**: lapas 2 yra tuščias, tad kamuoliukas, kurio vertė 30, įdedamas į lapą 2. Viršūnė  $P[2] = 5$  yra užimta, tad kamuoliukas lieka viršūnėje 2. Grąžinama reikšmė `true`.
5. **insert(1, 25)**: lapas 1 yra užimtas, tad kamuoliukas nebus įdėtas į viršūnę 1. Grąžinama reikšmė `false`.
6. **insert(0, 20)**: lapas 0 yra tuščias, tad kamuoliukas, kurio vertė 20, įdedamas į lapą 0. Viršūnė  $P[0] = 4$  yra tuščia, tad kamuoliukas pajuda į viršūnę 4. Kadangi viršūnė  $P[4] = 6$  yra užimta, kamuoliukas daugiau nejuda. Grąžinama reikšmė `true`.

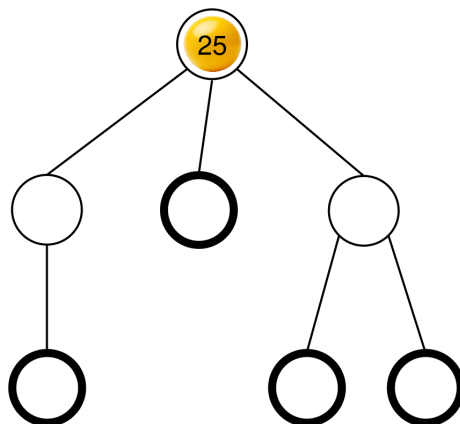
Gautas kamuoliukų išdėstymas pavaizduotas sąlygoje anksčiau.

Toliau procedūra gali kviesti:

```
collect()
```

Šio iškviatimo vykdymas aprašytas sąlygoje anksčiau. Grąžinamas masyvas yra  $S = [0, 10, 20, 30, 20]$ . Tuomet mašina vėl tuščia.

Tada procedūra gali iškviesti **insert(2, 25)**. Lapas 2 yra tuščias, tad kamuoliukas, kurio vertė 25, įdedamas į lapą 2. Šis kamuoliukas pajuda į viršūnę 5 ir tuomet į viršūnę 6, kur ir sustoja. Grąžinama reikšmė `true`. Gautas išsidėstymas mašinoje pavaizduotas paveikslėlyje žemiau.



Galiausiai procedūra gali iškviesti **collect()**, kuris grąžins  $S = [25]$  ir išvalys mašiną.

Yra du galimi teisingi atsakymo masyvai, kuriuos `find_structure` gali grąžinti:  $R = P = [4, 6, 5, 5, 6, 6]$  (kuris atitinka numeravimą  $L = [0, 1, 2, 3, 4, 5, 6]$ ) ir  $R = [5, 6, 4, 4, 6, 6]$  (kuris atitinka numeravimą  $L = [0, 1, 2, 3, 5, 4, 6]$ ). Šiame pavyzdyje  $K = 2$  ir  $B = 30$ , tad  $C = 32$ .

## Pavyzdinė vertinimo programa

Pradinių duomenų formatas:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Rezultatų formatas:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Čia  $Q$  žymimas ilgis masyvo  $R$ , grąžinto `find_structure`.